Internet Programming

CSE341
Lab 1

Introduction

What is Web Development

- Web development is the process of building websites and web applications that people use every day. For example:
  - When you watch videos on YouTube, that’s web development at work.
  - When you shop on Amazon, everything you see — from the product page to the checkout process — was built by web developers.
  - When you post a photo on Instagram, web technologies handle everything from the design you interact with to the servers that store your data.
  - Types of Web Development: Frontend, Backend, Full Stack

What is a Website?

- A website is a collection of many static or informational pages web pages interlinked together, and web pages are digital files that are written using HTML(HyperText Markup Language).
- Its main purpose is to present information.
- Usually built with HTML, CSS, and sometimes a little JavaScript.
- Content doesn’t change much unless the developer updates it.
- Examples:
    - Wikipedia
    - Online Newspaper (BBC News)
    - A personal portfolio site
    - A news site that mainly shows articles

What is a Web application?

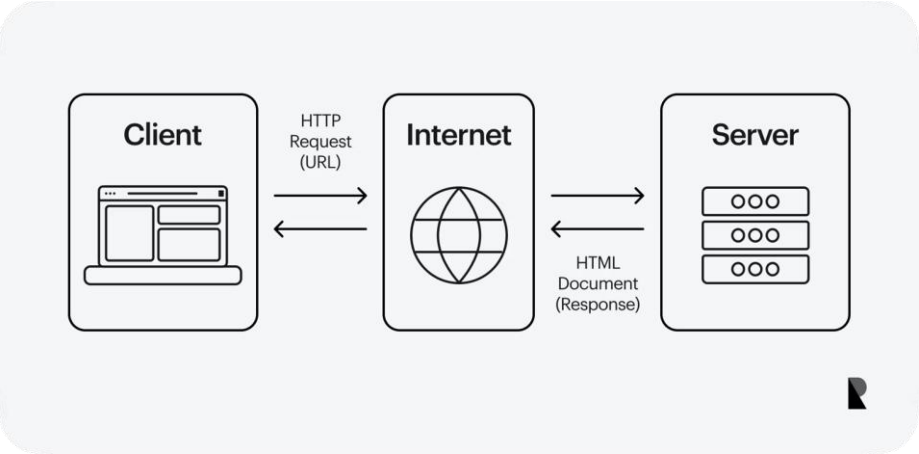
- A web app is a is a type of website that is an interactive program you access through your browser.
- Users can log in, enter data, interact, and get results.
- Built with both frontend (HTML, CSS, JS/React) and backend (servers, databases, APIs).
- Content changes depending on the user and their actions.
- Example:
  - Amazon.com is a website.
  - Inside it, the shopping cart, search, and payment system are web apps.

- Technically, all web apps are websites — but not all websites are web apps

What is a Web Server?

- A web server is a computer program (and sometimes the computer itself) that:
  - Stores websites and web apps (HTML, CSS, JS, images, backend code, etc.).
  - Listens for requests from users' browsers (the clients).
  - Sends back responses (like a web page, data, or an error message).

- Without a web server, your website would just sit on your laptop and no one else could access it.



Components of a Web Application

- Frontend (Client-Side)
  - The frontend (also called client-side) is the part of a website or web application that the user directly sees and interacts with in their browser.
  - It is called client-side because it runs on the user’s computer (the client), not the server.
  - Examples of Frontend in Action
    - On YouTube, the video player, search bar, and comments section you see are frontend.
    - On Instagram, the feed layout, the “like” button, and profile design are frontend.
    - On Amazon, the product listings, shopping cart icon, and checkout form are frontend.

Components of a Web Application (Cont.)

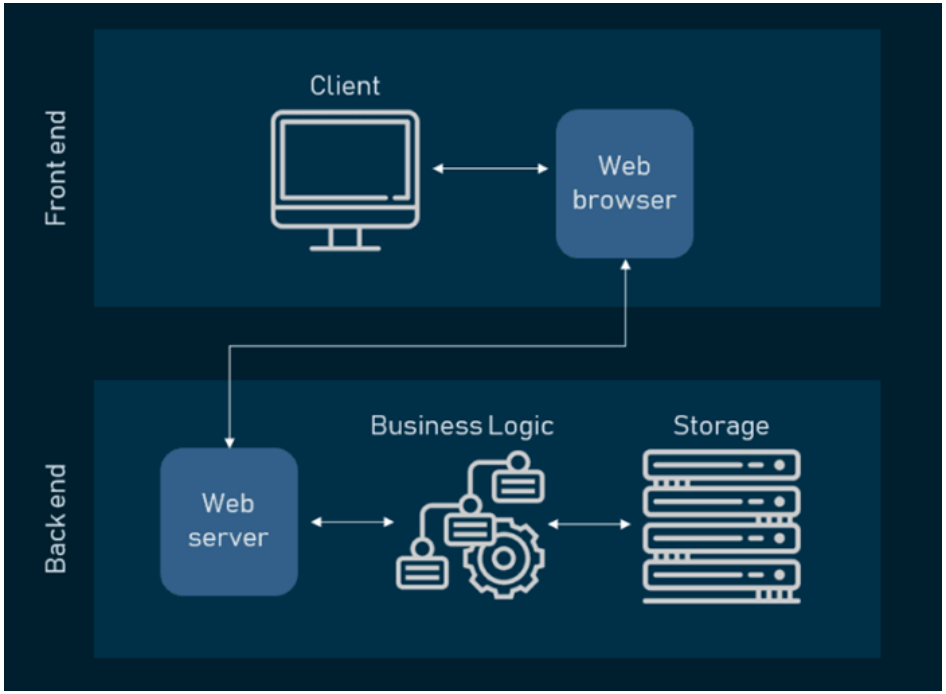
- Backend (Server-Side)
  - The backend (also called server-side) is the part of a website or web application that runs behind the scenes on the server.
  - It’s not visible to users in the browser.
  - It handles the logic, data, and communication that make the website work.
  - Examples of Backend in Action
    - On Instagram, when you log in, the backend verifies your username/password and loads your feed from the database.
    - On Amazon, the backend updates your shopping cart, processes your payment, and saves your order.
    - On Gmail, the backend sends, receives, and stores emails securely.

Components of Back End

- The backend itself has important components inside it:
  - Server → Listens to requests from the browser.
  - Application (Logic) → The code that processes those requests (e.g., “check if password is correct”).
  - Database → Stores and retrieves information (users, posts, orders, payments).

How Back-End and Front-End Work Together

- Frontend asks: “User clicked login, here’s the username and password.”
- Backend (Server) receives the request.
- Backend (Application) checks if it’s correct using the Database.
- Backend sends back a response (“Login successful!”).
- Frontend updates the page to show the logged-in user.



Course Overview

Course Overview

- This course will take you step by step from the very basics of how websites work to building interactive, modern web applications that people can actually use.

Course Overview (Cont.)

- Weeks 1->6 (Building the Frontend)

  - Week 1 -> Learn How the web works and Understand the big picture
  - Week 2 -> Use HTML and CSS to build static webpages
  - Week 3 -> Learn Javascript to add some interactivity to the website
  - Week 4 -> Learn advanced Javascript to manage more complex data on the page
  - Week 5 -> Learn React framework to make frontend more dynamic and scalable
  - Week 6 -> Learn more about React framework including Hooks and Lifecycle

- By the end of week 6, you will have the knowledge to build a fully functional frontend, but it won't save the data permanently so if you refresh the page everything will be reset.

Course Overview (Cont.)

- Weeks 8->10 (Building the Backend)
  - Week 8 -> Learn to fetch data from external resources using REST APIs, fetch, async/await to load on your front end using APIs and Promises
  - Week 9 -> Build your own server that listens for requests in the frontend using Node.js and Express
  - Week 10 -> Connect your backend to a database (MongoDB)

- By the end of week 10, we'll have a full-stack website: front-end + back-end + database

Course Overview (Cont.)

- Weeks 11->15 Deployment and Web Security
  - Week 11 -> Learn how to deploy your website using AWS to make your website live on the internet
  - Week 12 -> Learn about decentralized Apps
  - Week 13 -> Adding Real-time features like live chat or notifications using Sockets and WebSockets
  - Week 14 -> Learn the best practices to secure logins, payments and user data
  - Week 15 -> Learn how to integrate AI Agents to your website

How the Internet Works

How the Internet Works?

- The Internet is a massive, interconnected network of computers and devices all around the world. It's the physical infrastructure – the cables, satellites, routers, and servers – that allows any two devices to communicate with each other, no matter how far apart they are.
- Every single device connected to the Internet has a unique numerical label called an IP (Internet Protocol) address (e.g., 192.168.1.1 or 2001:0db8:85a3:0000:0000:8a2e:0370:7334 for IPv6). This is like a unique street address for your device on the global network.
- These addresses are essential because they tell the network exactly where to send and receive packets.

The World Wide Web

- While the Internet is the physical infrastructure, the World Wide Web (often just called "the Web") is an information system built on top of the Internet. It's the collection of interconnected documents and other resources (like web pages, images, videos) that you access via web browsers.
- At its core, the World Wide Web (or simply "the Web") is a vast, interconnected system of public web pages, websites, and web applications that you can access over the Internet.
- The Web is one of the many services (like email, file sharing, or online gaming) that uses that physical infrastructure to deliver its content.
- Internet = The physical roads and highways.
- Web = The cars, trucks, and traffic that travel on those roads, carrying specific types of cargo (web pages, videos, etc.).

Components of the WWW

- The Web is built upon a few key technologies and concepts that work together seamlessly:
  - Web Browsers:
    - Browsers are your client-side interface to the Web. They send requests to web servers, receive responses, and then interpret and display the content (HTML, CSS, images, videos, etc.) in a human-readable and interactive format on your screen.
    - Applications like Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.

Components of the WWW

- Web Servers
  - When your browser requests a web page, the web server's job is to locate that content and "serve" it back to your browser. They are always listening for incoming requests and responding with the appropriate files.
- HTTP/HTTPS
  - HyperText Transfer Protocol. This is the fundamental set of rules (protocol) that defines how messages are formatted and transmitted between web browsers and web servers. It dictates how a request is made and how a response is structured.

How the Web Works: A Step-by-Step Scenario

- Let's imagine you type https://www.facebook.com into your browser:
  - You Initiate a Request: Your browser (the client) sees the URL and understands you want to access the Facebook website.
  - DNS Lookup: Your computer first uses the Internet's Domain Name System (DNS) to translate www.facebook.com into its corresponding IP address (e.g., 104.18.11.134). This tells your computer where to find the Facebook web server.
  - HTTP/HTTPS Request: Your browser then constructs an HTTP (or HTTPS) request message. This message asks the Facebook web server for the homepage.
  - Internet Transport: This request message, broken into packets, travels across the Internet (through your home network, your ISP, various routers, potentially undersea cables, etc.) until it reaches the Facebook web server at its IP address.
  - Server Processes Request: The Facebook web server receives the request. It locates the requested files (typically an index.html file, along with associated CSS, JavaScript, and images).
  - HTTP/HTTPS Response: The web server sends back an HTTP (or HTTPS) response message. This message contains the HTML code for the homepage, along with instructions for your browser to fetch the CSS, JavaScript, and image files.
  - Content Delivery: Your browser receives the HTML. As it parses the HTML, it discovers references to CSS files, JavaScript files, and images. It then makes separate HTTP/HTTPS requests for each of these resources. The server sends those back.
  - Browser Rendering: Once your browser has all the necessary HTML, CSS, and JavaScript, it:
  - Interprets the HTML to understand the structure of the page.
  - Applies the CSS rules to style the page (colors, fonts, layout).
  - Executes the JavaScript to add interactivity and dynamic elements.

Git & Github

What is Git?

- Git is a free and open source Version Control System
- Imagine you're writing a really important document, and you keep saving different versions: "document_final.docx," "document_final_v2.docx," "document_final_really_final.docx," "document_final_for_real_this_time.docx." Sound familiar? That's what people did before VCS, and it was a recipe for disaster!

What is a Version Control System

- A Version Control System (VCS) is a software tool that helps a team of software developers (or anyone working with files) manage changes to source code over time. It keeps a complete history of every modification made, allowing you to:
  - Track Changes: See exactly who changed what, when, and why.
  - Revert to Previous Versions: Easily undo mistakes or go back to an earlier, stable state of your project.
  - Collaborate Seamlessly: Allow multiple people to work on the same files simultaneously without overwriting each other's work.
  - Branch and Merge: Experiment with new features in isolated "branches" without affecting the main project, and then seamlessly integrate those changes back when they're ready.

Key Concepts in VCS

- Repository (Repo): This is the central storage location where all the files, along with their complete history, are kept. Think of it as the project's master archive.
- Commit: A "commit" is a snapshot of your changes at a specific point in time. When you commit, you're essentially saying, "I'm done with this set of changes, save them to the history!" Each commit usually includes a message explaining what was changed.
- Branch: A branch is an independent line of development. Developers create branches to work on new features or bug fixes without disturbing the main codebase.
- Merge: Once work on a branch is complete and tested, it can be "merged" back into the main branch, integrating the new changes into the project.

What is Github?

- GitHub is a repository hosting service that provides a web-based platform for storing, managing, and collaborating on Git repositories. It's where your project's source code, its entire version history, and all the tools for team collaboration (like pull requests, issue tracking, and code review) live in the cloud.
- Git is the underlying technology for version control. It's the software that tracks changes, allows branching, and manages the history of your code. You use Git commands locally on your computer.
- GitHub (a repository hosting service) provides a remote home for your Git repositories. It's where you "push" your local Git changes to share them with others, back them up, and use GitHub's collaborative features.

GitHub

Practical Work

Setting Up Git & Github

Installing Git

1. For windows: Go to https://git-scm.com/downloads/win and download the appropriate installer (usually 64-bit for windows setup) and run it.
2. After installation, open Git Bash (comes with Git) and verify your installation using the command: git --version

1. Configure Git with your identity because it needs to know who you are when you make changes, this information will be attached to your commits. Type in Git Bash the following commands:
    i. git config --global user.name “Your Name”
    ii. Git config -- global user.email “your.email@example.com”

    (email should match your github account)

Creating a Repository on Github

1. Open your web browser and navigate to Github.com.
2. If you don’t have an account, sign up for free.
3. After creating your Github Account, go to your Profile -> Repositories -> you will find a green button “New”. Click to create your repository.
4. Fill in the appropriate information for your Github Repo such as its name, description, and add a readme file.

Connecting your local Git to your Github Repo

1. Create a folder for you project in your file directory as appropriate and Navigate to it. Right Click -> Open Git Bash Here. (or navigate from git bash accordingly)
2. The next step is cloning your github Repo. There are two ways to clone your repo on Github and they both differ on how your computer authenticates with Github when you pull or push changes:
    a. HTTPS: When you clone a repo using HTTPS, every time you perform an operation that interacts with Github, you'll be prompted to enter Github your username and password
    b. SSH: SSH uses keys for authentication. You generate a pair of keys on your local machine (a private key on your computer) and a public key (which you will upload to your Github account). You only set up once and after that authentication is done on its own.

Connecting your local Git to your Github Repo (Cont.)

1. We will use SSH to authenticate with the Github Repo.
2. First , generate the key SSH key pair in Git Bash using the command:
   i. ssh-keygen -t ed25519
3. Press enter to save the key in default path. Provide a passphrase if you wish (for this tutorial I did not provide one)

```
Mazen@LAPTOP-BCNURDLE MINGW64 ~
$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/Mazen/.ssh/id_ed25519):
Created directory '/c/Users/Mazen/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/Mazen/.ssh/id_ed25519
Your public key has been saved in /c/Users/Mazen/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:Aq4wQqMPRbUuRfD8rXHhUKu2rrv4flYsI2J6JceeFYA Mazen@LAPTOP-BCNURDLE
The key's randomart image is:
+--[ED25519 256]--+
|  o+o   .        |
| .E+.. . .       |
| o. *.. o        |
|o..+ o.= .       |
|* ..o *oS        |
|.=+o=.+*o        |
| oo* +o+         |
|. ..o.o          |
| ..o**.          |
+----[SHA256]-----+
```

Connecting your local Git to your Github Repo (Cont.)

1. Now the next step is to copy the public key in order to upload it to Github. So use the command:

   i. cat ~/.ssh/id_ed25519.pub

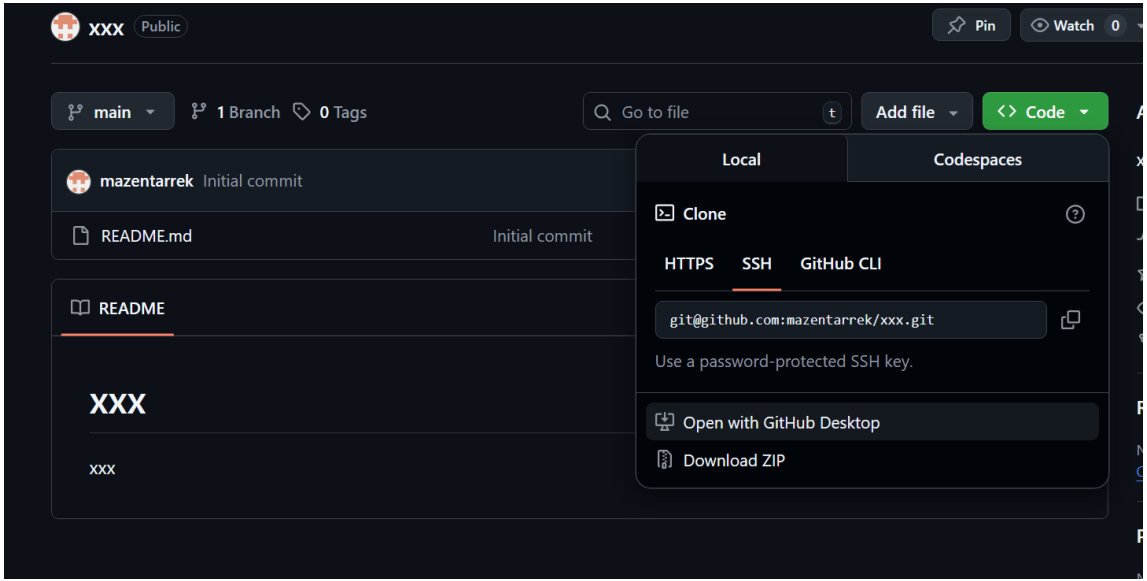
   To open the ssh key file and copy the key.

```
Mazen@LAPTOP-BCNURDLE MINGW64 ~
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGrhYB7OTH8PN2ajBnIqzmORJ/OtjuvQVqIs8K/Inq6g
 Mazen@LAPTOP-BCNURDLE
```

1. On Github go to Settings -> SSH and GPG Keys -> New SSH key -> Add a title to you SSH key like (My laptop -ed25519) -> Paste the copied SSH key

Connecting your local Git to your Github Repo (Cont.)

1. Now navigate to your created Repo -> and click the green button <> Code -> Local -> SSH -> and copy the link



Connecting your local Git to your Github Repo (Cont.)

1. Open your git bash terminal in the file directory you created for your project and type the command:
    i. Git clone (your copied ssh link)

```
$ git clone git@github.com:mazentarrek/xxx.git
Cloning into 'xxx'...
The authenticity of host 'github.com (140.82.121.4)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Lab task

1. Try creating a text file in your local file directory where the github repo is cloned
2. Open Git bash in the file directory path
3. Type git status (Show which files are new, modified, or staged)
4. Type git add . (This command stages all your changes in the project)
5. Type git commit -m “Descriptive Message” (This command saves the staged changes with a descriptive message)
6. Type git push origin main (This command uploads commits on main branch)